Midterm exam — Version A — ANSWER KEY

TC1037 Algorithmic Thinking · Group 510 · Tecnológico de Monterrey, Campus Querétaro

Friday 23 October 2026 · 17:35–18:55 · Room 4206

Name: _____ **Matrícula:** _____

80 minutes · 100 points · individual · on paper. No computer, phone, notes or calculator. Write in pen or dark pencil; code must be legible and indented. **Part I** (30 pts): write *exactly* what Python prints — every line, in order. **Part II** (70 pts): write the program. Suggested time: 15 minutes for Part I, 65 for Part II.

Question:	1	2	3	4	5	6	7	8	9	10	11	Total
Points:	5	5	5	5	5	5	23	23	3	20	1	100
Score:												

Part I — Read and predict **(30 points)**

1. **Arithmetic operators.** What does Python print? [5]

```
print(7 + 8 // 3 * 2 - 9 % 4 ** 2)
```

Solution: `2`

2. **Boolean operators.** What does Python print? [5]

```
print(not 3 > 2 or 4 == 4.0 and 5 < 5)
```

Solution: `False`

3. **Conditional structure.** What does Python print? [5]

```
x = 12
y = 7
if x % y > 4:
    print("A", x - y)
elif x > y and y % 2 == 1:
    print("B", x + y)
else:
    print("C")
print("end")
```

Solution:
```
A 5
end
```

4. **Functions with a `for` loop.** What does Python print? [5]

```python
def f(a, b):
    for i in range(b):
        a = a + i
    return a

def main():
    a = 3
    b = 4
    c = f(b, a)
    print(a, b, c)

main()
```

Solution: `3 4 7`

5. **The while loop.** What does Python print? [5]

```python
n = 1
total = 0
while n < 20:
    total = total + n
    n = n * 3
print(n, total)
```

Solution: `27 13`

6. **The for loop.** What does Python print? [5]

```python
for i in range(10, 0, -3):
    if i % 2 == 0:
        print(i)
    else:
        print(-i)
```

Solution: `10`
`-7`
`4`
`-1`

Part II — Write the program **(70 points)**

Write **one** Python program with the following parts. All `input()` and all validation happen in `main()`; the two functions **return** their result and never print or read. **Function 7 must use a `while` loop and function 8 a `for` loop** — each is worth 23 points. Write each function's purpose in a one-line comment above it, and under each loop a one-line comment saying why it stops (`while`) or how many times it runs (`for`).

7. **`while loop`** The function `count_digit(n, d)` receives a non-negative integer `n` and a digit `d` (0–9) and **returns** how many times `d` appears among the digits of `n`. Use a `while` loop with `%` and `//`; no strings, no lists. Careful: the number `0` has one digit, and it is `0`. [23]

   Examples: `count_digit(1212, 2)` → `2` `count_digit(7000, 0)` → `3` `count_digit(5, 3)` → `0` `count_digit(0, 0)` → `1`

8. **`for loop`** The function `alternating_sum(n)` receives an integer $n \geq 1$ and **returns** $1 - 2 + 3 - 4 + \cdots \pm n$ (odd terms added, even terms subtracted). Use a `for` loop. [23]

   Examples: `alternating_sum(1)` → `1` `alternating_sum(2)` → `-1` `alternating_sum(5)` → `3` `alternating_sum(6)` → `-3`

9. The function `menu()` that displays: `1. count_digit  2. alternating_sum  3. Exit` [3]

10. The function `main()` that, inside a `while` loop: shows the menu, reads the option, and [20]
    - option 1: reads n (must be 0 or more) and d (must be between 0 and 9); if any value is out of range displays a clear message, otherwise calls `count_digit` and displays the result with a label;
    - option 2: reads n (must be 1 or more); same rule, calling `alternating_sum`;
    - option 3: displays `Goodbye.` and the program ends;
    - any other option: displays `Invalid option.` and the program ends.

    Use `if`/`elif`/`else` to choose the option.

11. The last line of the script calls `main()`. [1]

Technical requirements — 2 points off for each one missing: meaningful variable names in Python style · a clear prompt for every input · readable, labelled output.

Reference solution

```python
# Reference solution — Midterm exam — Version A

def count_digit(n, d):
    if n == 0:
        if d == 0:
            return 1
        return 0
    count = 0
    while n > 0:
        if n % 10 == d:
            count = count + 1
        n = n // 10
    return count
```

```python
def alternating_sum(n):
    total = 0
    for i in range(1, n + 1):
        if i % 2 == 1:
            total = total + i
        else:
            total = total - i
    return total


def menu():
    print("1. count_digit")
    print("2. alternating_sum")
    print("3. Exit")


def main():
    keep_going = True
    while keep_going:
        menu()
        option = int(input("Option: "))
        if option == 1:
            n = int(input("Number n (0 or more): "))
            d = int(input("Digit d (0 to 9): "))
            if n < 0 or d < 0 or d > 9:
                print("Invalid input: n must be 0 or more and d between 0 and 9.")
            else:
                print("The digit", d, "appears", count_digit(n, d), "times in", n)
        elif option == 2:
            n = int(input("n (1 or more): "))
            if n < 1:
                print("Invalid input: n must be 1 or more.")
            else:
                print("Alternating sum:", alternating_sum(n))
        elif option == 3:
            print("Goodbye.")
            keep_going = False
        else:
            print("Invalid option.")
            keep_going = False


main()
```

Part I Q4: `f(b, a)` crosses the arguments (`a`=4, `b`=3 inside) and `range(3)` is 0, 1, 2 → $4 + 0 + 1 + 2 = 7$; main unchanged → `3 4 7`. Common wrong answers: `3 4 9` (no swap: `a`=3, `range(4)`) and `3 4 10` (`range(3)` read as 0–3).

What to look for. `count_digit(0, 0)` must return 1: a plain `while n > 0` returns 0 — that is the trap, worth the difference between Excelente and Muy bien. `alternating_sum`: `range(1, n)` drops the last term (`(5)` gives -1). Accept `total += (-1)**(i+1) * i`. Validation of `d` in `main`, not in the function.

How Part II is graded (the academia's rubric)

Each part of the program (the two functions, `menu`, `main`, the call) is scored on this scale, as a percentage of its points:

Excelente 100%	**Muy bien 85%**	**Regular 70%**	**Deficiente 50%**	**Muy deficiente 20%**	**Sin evidencia 0%**
No syntax errors; works correctly, including the boundary cases in the examples.	Correct approach; minimal logic or syntax slips.	At most two syntax errors and/or small logic errors; the approach is closely related to what was asked.	Approach loosely related, or related but with more than two syntax errors.	Code unrelated to the problem.	No code.

Part I: 5 points per question, all or nothing — every line of the output, in order. Technical requirements: -2 each, from Part II.